Pseudo-Classes & Responsive Design

States, Positions & Screens — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser**, **VS Code**, and **DevTools** open beside you. For every task, **attempt and write your answer *before* we discuss it** — your first guesses matter more than being right.

Part 1 — Pseudo-Classes: Styling States and Positions

TRY THIS · VS CODE

Create `pseudo-classes.html` inside an appropriate folder. Add a plain `<a href="#">Visit Page</a>` — no CSS at all. Move your mouse over it, then click and hold it (don't release).

Q1. Did the cursor change when you hovered over it? Did anything happen visually while you were clicking and holding — even though you haven't written a single line of CSS? Where do you think that's coming from?

TRY THIS · VS CODE

Add a `<button>Click Me</button>` with just a `background-color` and `color`. Add: `button:hover { background-color: darkblue; }` Now, move your mouse over the button without clicking.

Q2. What happened when your mouse entered the button, and when it left? Now you're writing the rule yourself instead of relying on the browser's default — what do you think `:hover` is reacting to?

TRY THIS · VS CODE

Add `button:active { background-color: black; }`, keeping `:hover`. Click and hold the button.

Q3. What color shows while actively pressing, versus just hovering? What's the real difference between `:hover` and `:active`?

Q4. Try the `scale` property with values such as `1.25`, `0.75`, and `1.0`, and reason about what it does.

TRY THIS · VS CODE

Build a `<ul>` with 10 `<li>` items. Using only what you already know (`id` or `class`), try to style just the 3rd, 5th, and 7th items orange.

Think: If this were 100 items in the list and you wanted every other item styled (some color or anything else) —

Q5. What would you do to make that work? What CSS would come to help?

TRY THIS · VS CODE

Remove your current CSS for the 3rd / 5th / 7th items. Add this rule: `li:nth-child(2) { color: red; }`. See if that worked.

Q6. Which item turned red with `nth-child(2)`? What does the `2` refer to?

Q7. Now try `li:nth-child(even) { color: orange; }`. Which items get styled this time?

TRY THIS · VS CODE

Add this code to the end of your body (or create a new file to start fresh):

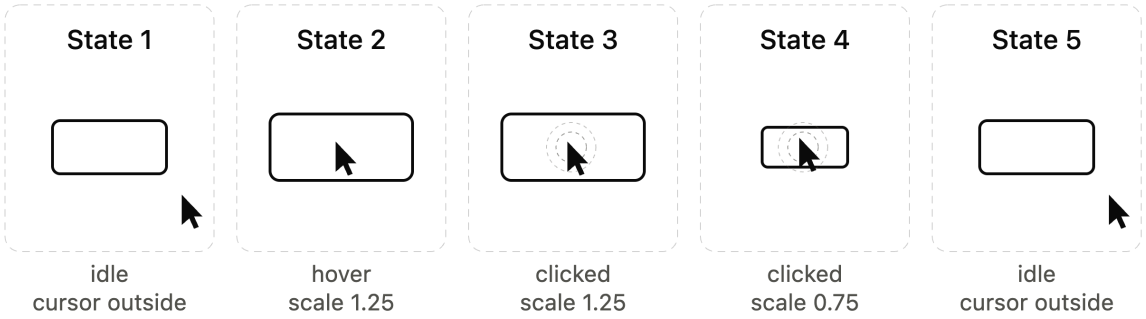
```
<body>
    <div>
        <h2>Title One</h2>
        <h2>Title Two</h2>
        <p>Text One</p>
        <p>Text Two</p>
    </div>
</body>
```

Add `p:nth-child(3) { color: green; }` and check what turns green. Then replace it with `p:nth-of-type(2) { color: green; }`.

Q8. What is different about `nth-of-type(2)` as compared to `p:nth-child(4)`?

TRY THIS · REFERENCE

The image below shows the user flow of clicking on a button:



Q9. Can you implement this? Write the CSS here for the button.

```
button {


}
button:hover {


}
button:active {


}
```

Part 2 — Responsive Design: Media Queries & Display

TRY THIS · BROWSER

Open one of your earlier files — `profile-card.html` works well. Open Developer Tools and then shrink the viewport window very narrow, like a phone screen (~375px).

Q10. Does your page still look good at that width? What looks broken, cramped, or awkward? Can one single CSS rule work for both desktop and mobile view?

TRY THIS · VS CODE

In any file, add: `@media (max-width: 600px) { body { background-color: yellow; } }`
Resize your browser window slowly across the 600px mark (check the width in DevTools).

Q11. Inspect the `<body>` in DevTools and look at the CSS rules attached to it. Compare the styles applied at a width greater than 600px versus less than 600px.

TRY THIS · VS CODE

Change the selector from `@media (max-width: 600px)` to `@media (width <= 600px)`. Test the same resize.

Q12. Did anything behave differently? Compare both syntaxes for writing width-specific CSS.

TRY THIS · VS CODE

Now change it to `@media (min-width: 600px) { ... }` instead. Test the same resize.

Q13. What do you think `min-width` lets you target, compared to `max-width`?

TRY THIS · VS CODE

Add two `<span>` elements and two `<div>` elements, each with a `background-color`.

Q14. What's different about how these two tags behave by default? How are they arranged in the layout?

TRY THIS · VS CODE

Add `span { display: block; }` and `div { display: inline; }`.

Q15. What do you think `display: block` and `display: inline` actually control?

TRY THIS · VS CODE

Pick any one element on your page and add `display: none;` to it.

Q16. What happened to that element on the page? Is it just invisible, or is the space it used to take up still there?

Q17. Now imagine combining this with `@media` — how could you use `display: none;` to hide something only on narrow screens?

TRY THIS · VS CODE

Reopen `profile-card.html`. If it's not already there, then add `<meta name="viewport" content="width=device-width, initial-scale=1">` inside `<head>` — real phones need this line to size the page using their actual screen width instead of a default fake width. Then add: `@media (max-width: 500px) { .card { width: 90%; } }` Test by shrinking the browser below 500px.

Q18. What did changing `width: 400px` to `width: 90%` inside the media query actually do?

Q19. Given that you need to design a landing page for a company, how will you approach the design? Will you start mobile-first or desktop-first, and why?